

Microduck 硬件图集

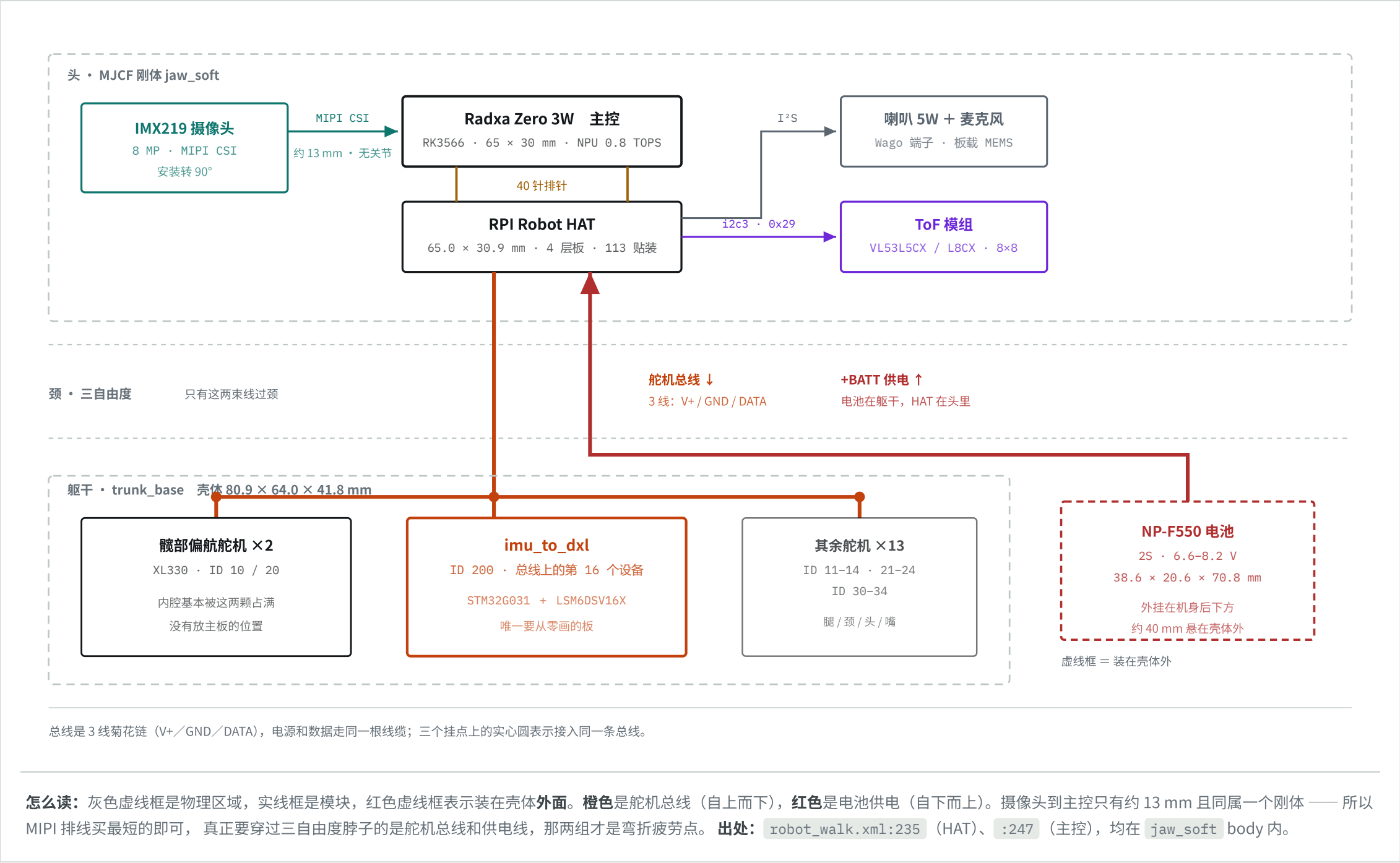
五个模块、两块自制板、一条总线。所有引脚、地址与寄存器均取自官方开源硬件工程与运行时源码，未经实物验证。

舵机总线 · 半双工 TTL 1 Mbps +BATT 6.6-8.2 V +5V / +3V3 I²C · 400 kHz MIPI CSI I²S 音频

总线设备 16 15 舵机 + 1 块 IMU 板	控制周期 20 ms 50 Hz，一读一写	自制板 2 HAT 已开源 / IMU 板要自绘	策略网络 775 KiB ×9 个 ONNX
--	--	---	---

01 东西装在哪

最容易搞错的一条：**主控不在躯干，在头里**。摄像头、主控板、HAT 三者同属 MJCF 里的 `jaw_soft` 刚体，摄像头距主控板中心约 13 mm，中间没有任何关节。躯干内腔基本被两颗髌部偏航舵机占满，电池则外挂在机身后下方。

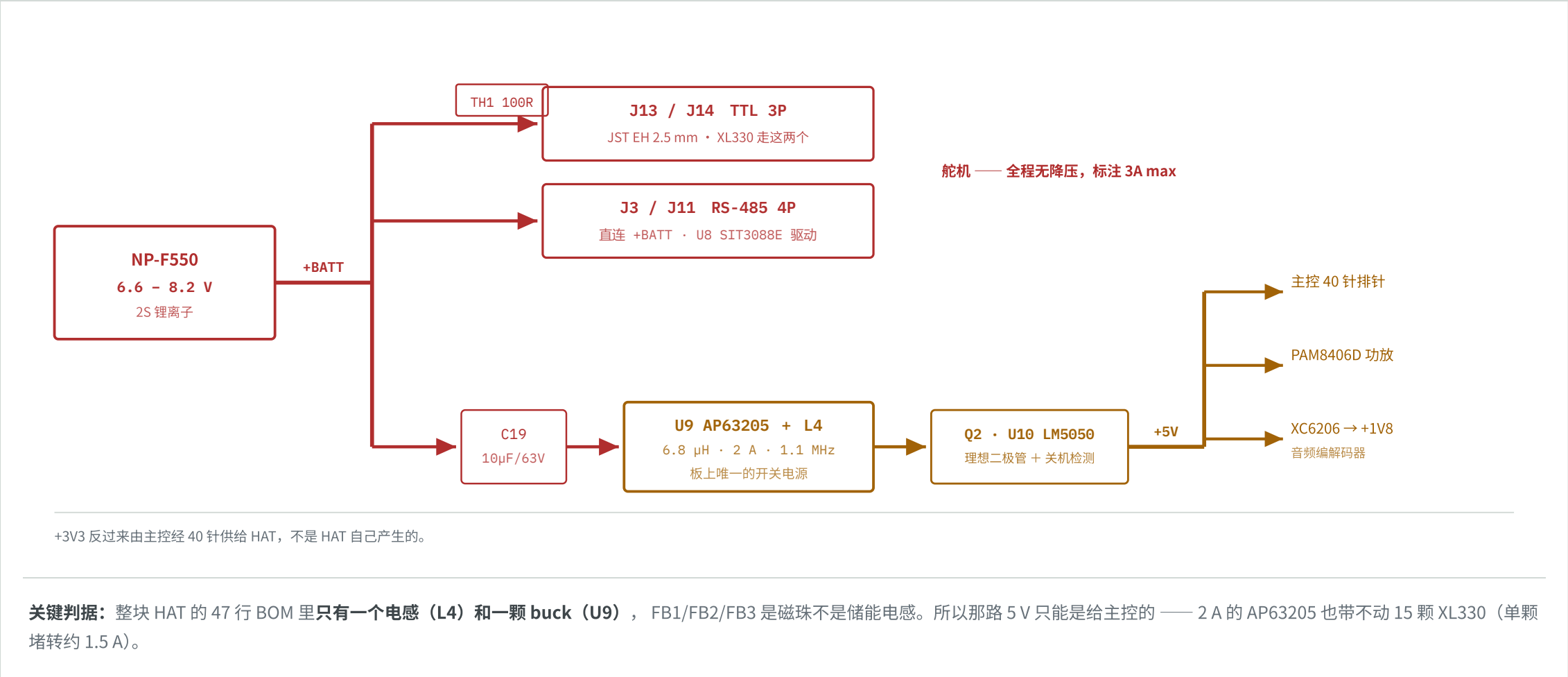


买之前先看这条

因为主控在头里、槽位只有 65 × 30 mm，任何更大的板子都塞不进去 —— 立创泰山派是 70 × 45 mm，同为 RK3566 但装不下，上面还压着一块 HAT。

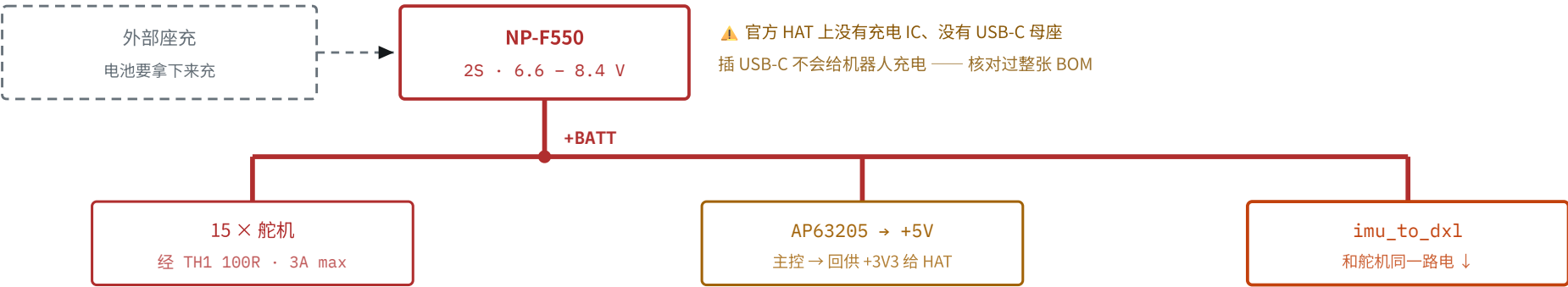
02 电池两路走，舵机吃原压

这是整块 HAT 上最反直觉的一件事：**舵机接的是电池原压，没有任何降压**。XL330 官方额定 3.7–6.0 V、推荐 5.0 V，而这里给它 6.6–8.2 V。这不是笔误，是官方主动的选择——换来更大的堵转力矩，代价是发热与寿命。



供电、充电与过压保护

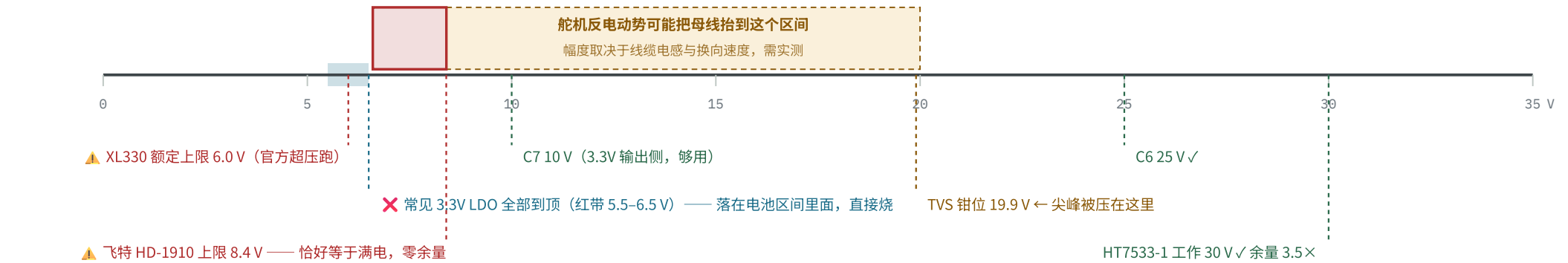
充电 —— 板上没有充电电路



imu_to_dxl 的输入保护链 —— 它和舵机共用母线，会吃到反电动势



电压窗口 —— 谁是瓶颈，一眼看出



三道防线：宽压 LDO (已选) 挡住持续过压 · TVS (待加) 钳住瞬态尖峰 · PPTC (待加) 让本板故障不拖垮整条总线。串联二极管挡的是反接，不解决正向过压。

怎么读：数轴上方是实际会出现的电压，下方是各器件的耐压上限。只要下方的线落在上方区间之内，那个器件就是瓶颈。常见的 3.3V LDO 全部落在电池区间里面 —— 直接烧。⚠ 反电动势的幅度是示意，不是实测 —— 取决于线缆电感与换向速度，到货要拿示波器量。

这解释了固件里的电压自适应

因为舵机直吃电池，电量从满到亏的过程里同一条指令产生的力矩差很多。运行时用 $\text{scale} \times (7.4 / \text{实测电压 EMA})$ 做补偿，不然策略在不同电量下表现会飘。

03 一条总线挂 16 个设备

整个方案的主论点在这里：**15 颗舵机菊花链**在一条三线总线上，而 IMU 也伪装成总线上**的一个设备**。于是主控一个周期只发一次 `sync_read`，就同时拿回身体姿态和 15 个关节角度——时间戳天然对齐，不需要在软件里对付两条不同数据通路之间的抖动。



20 MS 预算比想象中紧

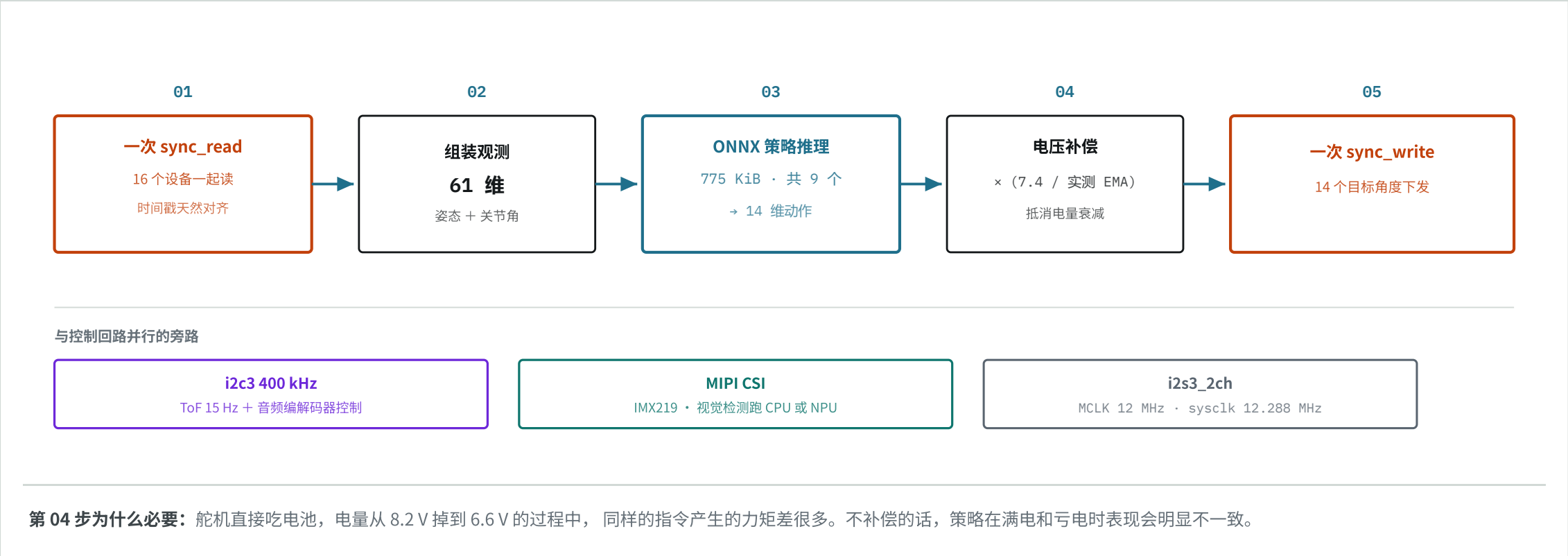
XL330 出厂 `return_delay_time = 250`，即**每个设备 500 μs 的总线转向延迟**。

源码原文：「Across 16 devices that is **8 ms per tick — 40% of a 20 ms budget**」。官方的解法是把这个寄存器写 0。

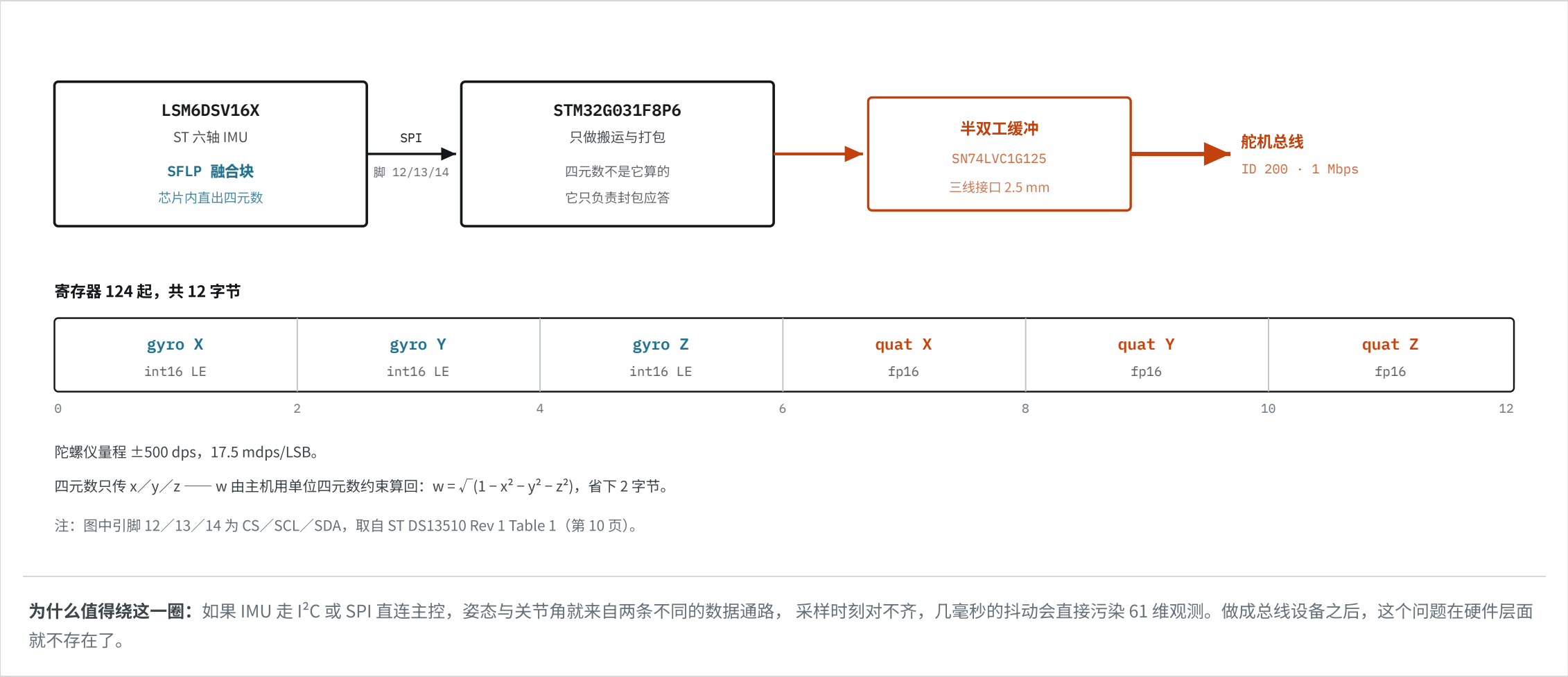
换任何舵机，这一步都要重做并实测——这不是某一家的问题。

04 一个周期，20 毫秒

控制回路只有五步，按顺序发生。这里的编号是真实的时序，不是排版装饰。



它的全部工作只有三件：读 IMU、把数据打包成 12 字节、在主机发起 `sync_read` 时把这 12 字节吐出去。官方没有开源这块板，但协议与寄存器布局可以从运行时源码完整还原——代码即规格书。



同时适配飞特与原厂：硬件不用改

两种协议的物理层完全相同——半双工单线 TTL、1 Mbps、三线、2.5 mm 间距接头。差别只在包格式：Dynamixel V2 是 FF FF FD 00 + CRC-16，飞特 STS/SCS 是 FF FF + 取反和。

所以硬件画一版，固件分两套解析，甚至可以上电监听总线、看第三字节是不是 FD 来自动识别，插上就能用。

06 40 针占用与关键地址

引脚 / 总线	用途	LINUX 设备	出处
GPI014 / GPI015	舵机总线 Tx / Rx	/dev/ttyS2 @ 1 Mbps	robotd.toml
GPI002 / GPI003	I ² C —— IMU、音频编解码器、Stemma	i2c3 @ 400 kHz	i2c3-pihat.dts
GPI018 / 19 / 20 / 21	音频 PCM 时钟 / 帧同步 / 输入 / 输出	i2s3_2ch	aic3104-i2c3.dts
GPI006	关机检测	—	HAT 原理图
GPI022	HAT 上 BMI088 的中断脚 —— 软件不用	—	i2c3-pihat.dts
MIPI CSI	IMX219 摄像头，安装转 90°	radxa-zero3-rpi-camera-v2	setup-board.sh
i2c3 · 0x18	音频编解码器 TLV320AIC3104	—	i2c3-pihat.dts
i2c3 · 0x29 / 0x52	ToF，两个地址都试	—	tof/src/main.rs
i2c3 · 0x19 / 0x68	BMI088 —— 贴了片但软件标注 dormant	—	i2c3-pihat.dts
总线 ID 200	imu_to_dxl，寄存器 124，长 12 字节	—	duck-control/model.rs

这张表就是核对清单

Microduck 的软件是开源的，硬件上每一个地址都能在代码里找到对应。对不上，就是有一边错了 —— 这是复刻时最省事的验证手段。

07 两条路线：原厂 XL330 与 飞特 STS

「把 IMU 做成总线上的第 16 个设备」这个架构，在飞特上同样成立 —— 因为飞特 STS/SCS 有自己的批量读指令 SYNC READ （0x82），用法与 Dynamixel 完全一致。

而且两边的应答机制是一样的。Protocol V2 确实有个能把所有应答拼成一个包的 fast sync read （0x8A），但官方运行时没有用它 —— 全库检索 fast_sync_read / 0x8A 零命中， duck-control/src/bus.rs:236 调的是普通的 sync_read_raw_data。这也和源码里「500 μs per device × 16 devices = 8 ms」自洽 —— 如果应答拼成一个包，per-device 的延迟就不会乘以 16。



协议差异一览

项	DYNAMIXEL V2 · XL330	飞特 STS/SCS
物理层	完全相同 —— 半双工单线 TTL、3 线（V+／GND／DATA）、1 Mbps	
接头间距	JST EH 2.5 mm	5264, 同为 2.5 mm
包头	FF FF FD 00	FF FF
校验	CRC-16	~sum（取反和）
批量读	sync read（官方实际用的） 另有 fast sync read 0x8A, 未被使用	sync read 0x82
应答方式	相同 —— 每设备一个独立状态包	
总线转向 / 周期		17 ⚠
⚠ 转向次数是协议推论（1 条指令 + 16 个应答），非实测		
出厂应答延时	250 = 500 μs／设备，须写 0	默认 0，且 STS 上不起作用
ID 范围	0 - 252	0 - 253（寄存器 5）
波特率设定	寄存器 baud_rate = 3	寄存器 6，0 = 1M

飞特侧还有一个可优化的寄存器

STS 内存表第 8（0x08）号 **Response Level**，默认 **1** —— 「所有命令都应答」。
改成 **0** 之后只有 read 与 ping 才回包，普通写指令不再占用总线时间。

这与官方对 XL330 把 return_delay_time 写 0 是同一类动作：**出厂默认值是给通用场景的，高频控制回路要自己调。**

换成飞特，各模块受什么影响

模块	影响	说明
主控	不变	与舵机协议无关
摄像头 / ToF	不变	走各自独立的接口
HAT	完全不用改	飞特与 XL330 物理层完全相同 —— 三线半双工 TTL、1 Mbps、2.5 mm 间距接头。HAT 上的 J13／J14 是 TTL 3P 口，直接插飞特舵机即可 ，一根线都不用动
imu_to_dx1	硬件架构不变	物理层相同，所以硬件画一版即可；固件里按包头第三字节是不是 FD 自动识别协议，两套解析分支。连接器统一到 2.5 mm 间距，两边通用
策略	大概率要重训	舵机换了意味着力矩曲线与质量都变了，官方九个 ONNX 不能直接套用。这是整条路线上最大的一笔工作量，与协议无关

另一个问题：这块 HAT 到底要不要做

这与用哪家舵机**无关**，是纯成本判断。HAT 是 4 层板加贴片，打样几百元起且有起订量，而板上**约一半电路是音频**（编解码器、功放、板载麦克风、喇叭端子）。

如果你不做语音功能，那半块板就白打了 —— 此时一块二十几元的总线转接板加一个降压模块，就顶掉了「舵机总线」和「配电」这两件真正必需的功能，ToF 直接挂 i2c3。**代价是失去板载音频与 Pi HAT+ 身份识别。**

画板子之前先做这件事

上面的转向次数是**协议事实**，但换算成毫秒是**估算**。真实数字取决于三态缓冲切换速度、包间隙与实际读取的寄存器长度 —— 只有拿实物测过才知道。

舵机到货第一件事：接上调试板，用两种读法各跑一遍，量出 16 个设备一轮的实际耗时。**这个数不出来，板子上的架构决策就是空转。**

基于官方公开的 MJCF 模型、STL 网格、开源硬件工程（elec_RPI_Robot_HAT，Apache-2.0）与运行时源码推导，**未经实物验证**。图中所有引脚、地址、寄存器与器件位号均可在上述来源中复核。